Prolog Practical Assignment

Name: ARJUN KUMAR MAHATO **ERP No.:** RU-25-10269 **Section:** 3B

Subject: Artificial Intelligence and Machine Learning

Practical 1: Create a simple Knowledge Base using facts, predicates, clauses, and goals (Student-Course Database).

```
% Student-Course Knowledge Base
student(mohit).
student(ananya).
student(rahul).

course(ai_ml).
course(prolog).
course(data_structures).

enrolled(mohit, ai_ml).
enrolled(mohit, prolog).
enrolled(ananya, ai_ml).
enrolled(rahul, data_structures).

teaches(dr_sharma, ai_ml).
teaches(ms_gupta, prolog).
teaches(mr_khan, data_structures).

classmate(Student1, Student2) :-
    enrolled(Student1, Course),
    enrolled(Student2, Course),
    Student1 \= Student2.

student_teacher(Student, Teacher) :-
    enrolled(Student, Course),
    teaches(Teacher, Course).

% Sample goals:
% ?- enrolled(mohit, Course).
% ?- student_teacher(mohit, Teacher).
```

Practical 2: Implement arithmetic operations, Factorial, and Fibonacci Series using recursion.

```
% Arithmetic operations
add(A, B, Result) :- Result is A + B.
subtract(A, B, Result) :- Result is A - B.
multiply(A, B, Result) :- Result is A * B.
divide(_, 0, _) :-
    write('Division by zero is not allowed'), fail.
divide(A, B, Result) :-
    B =\= 0,
    Result is A / B.

% Factorial using recursion
factorial(0, 1).
factorial(N, Result) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, Partial),
    Result is N * Partial.

% Fibonacci series term using recursion
fibonacci(0, 0).
fibonacci(1, 1).
fibonacci(N, Result) :-
    N > 1,
    N1 is N - 1,
    N2 is N - 2,
    fibonacci(N1, F1),
    fibonacci(N2, F2),
    Result is F1 + F2.

% ?- factorial(5, F), fibonacci(8, Fib).
```

Practical 3: Perform append, reverse, member search, and list length operations.

```
% Append two lists
my_append([], List, List).
my_append([Head | Tail], List, [Head | Result]) :-
    my_append(Tail, List, Result).

% Reverse a list using an accumulator
my_reverse(List, Reversed) :-
    reverse_acc(List, [], Reversed).

reverse_acc([], Accumulator, Accumulator).
reverse_acc([Head | Tail], Accumulator, Reversed) :-
    reverse_acc(Tail, [Head | Accumulator], Reversed).

% Search for an element in a list
member_of(Item, [Item | _]).
member_of(Item, [_ | Tail]) :-
    member_of(Item, Tail).

% Find the length of a list
list_length([], 0).
list_length([_ | Tail], Length) :-
    list_length(Tail, TailLength),
    Length is TailLength + 1.

% ?- my_append([1,2], [3,4], X), my_reverse(X, R).
% ?- member_of(3, [1,2,3]), list_length([a,b,c], N).
```

Practical 4: Develop a timetable database and answer queries regarding faculty, subjects, and classrooms.

```
% timetable(Day, Time, Faculty, Subject, Classroom).
timetable(monday,    '09:00', dr_sharma, ai_ml,        lab_201).
timetable(monday,    '11:00', ms_gupta,  prolog,       lab_202).
timetable(tuesday,   '09:00', mr_khan,   data_structures, room_105).
timetable(wednesday, '10:00', dr_sharma, machine_learning, lab_201).
timetable(thursday,  '12:00', ms_gupta,  ai_ml,        room_107).
timetable(friday,    '10:00', mr_khan,   prolog,       lab_202).

faculty_schedule(Faculty, Day, Time, Subject, Room) :-
    timetable(Day, Time, Faculty, Subject, Room).

subject_schedule(Subject, Day, Time, Faculty, Room) :-
    timetable(Day, Time, Faculty, Subject, Room).

room_schedule(Room, Day, Time, Faculty, Subject) :-
    timetable(Day, Time, Faculty, Subject, Room).

% ?- faculty_schedule(dr_sharma, Day, Time, Subject, Room).
% ?- subject_schedule(prolog, Day, Time, Faculty, Room).
% ?- room_schedule(lab_202, Day, Time, Faculty, Subject).
```

Practical 5: Classify students as topper, average, and fail using rule-based reasoning.

```
% student(Name, Marks).
student(mohit, 92).
student(ananya, 78).
student(rahul, 55).
student(priya, 34).

topper(Student) :-
    student(Student, Marks),
    Marks >= 85.

average(Student) :-
    student(Student, Marks),
    Marks >= 40,
    Marks < 85.

fail(Student) :-
    student(Student, Marks),
    Marks < 40.

classification(Student, topper) :- topper(Student).
classification(Student, average) :- average(Student).
classification(Student, fail) :- fail(Student).
```

```
% ?- classification(mohit, Category).
% ?- topper(Student).
```

Practical 6: Implement Breadth-First Search (BFS) to find the shortest path in a graph.

```
% Undirected sample graph
edge(a, b).  edge(a, c).
edge(b, d).  edge(b, e).
edge(c, f).  edge(e, g).
edge(f, g).

connected(X, Y) :- edge(X, Y).
connected(X, Y) :- edge(Y, X).

% A queue entry is a path stored in reverse order.
bfs(Start, Goal, Path) :-
    bfs_queue([[Start]], Goal, ReversePath),
    reverse(ReversePath, Path).

bfs_queue([[Goal | Rest] | _], Goal, [Goal | Rest]).
bfs_queue([[Node | Rest] | Queue], Goal, Path) :-
    findall([Next, Node | Rest],
            (connected(Node, Next), \+ member(Next, [Node | Rest])),
            NewPaths),
    append(Queue, NewPaths, UpdatedQueue),
    bfs_queue(UpdatedQueue, Goal, Path).

% ?- bfs(a, g, Path).
```

Practical 7: Implement Greedy Best-First Search using a heuristic function to find a path in a graph.

```
% Undirected sample graph
edge(a, b).  edge(a, c).
edge(b, d).  edge(b, e).
edge(c, f).  edge(d, g).
edge(e, g).  edge(f, g).

connected(X, Y) :- edge(X, Y).
connected(X, Y) :- edge(Y, X).

% Estimated distance to goal g
heuristic(a, 6).  heuristic(b, 4).  heuristic(c, 5).
heuristic(d, 1).  heuristic(e, 2).  heuristic(f, 3).
heuristic(g, 0).

greedy_best_first(Start, Goal, Path) :-
    heuristic(Start, H),
    greedy_queue([H-[Start]], Goal, ReversePath),
    reverse(ReversePath, Path).

greedy_queue([_- [Goal | Rest] | _], Goal, [Goal | Rest]).
greedy_queue([_- [Node | Rest] | Queue], Goal, Path) :-
    findall(H-[Next, Node | Rest],
            (connected(Node, Next),
             \+ member(Next, [Node | Rest]),
             heuristic(Next, H)),
            Children),
    append(Queue, Children, Frontier),
    keysort(Frontier, OrderedFrontier),
    greedy_queue(OrderedFrontier, Goal, Path).

% ?- greedy_best_first(a, g, Path).
```

Practical 8: Implement the A* Search algorithm to find the optimal path using path cost and heuristic estimate.

```
% edge(From, To, Cost) for an undirected weighted graph
edge(a, b, 2).  edge(a, c, 4).
edge(b, d, 3).  edge(b, e, 6).
edge(c, e, 2).  edge(d, g, 5).
edge(e, g, 2).

connected(X, Y, Cost) :- edge(X, Y, Cost).
connected(X, Y, Cost) :- edge(Y, X, Cost).

% Admissible heuristic values for goal g
```

```
heuristic(a, 7).  heuristic(b, 6).  heuristic(c, 4).
heuristic(d, 5).  heuristic(e, 2).  heuristic(g, 0).

astar(Start, Goal, Path, TotalCost) :-
    heuristic(Start, H),
    astar_queue([H-node(Start, [Start], 0)], Goal, ReversePath, TotalCost),
    reverse(ReversePath, Path).

astar_queue([_-node(Goal, Path, Cost) | _], Goal, Path, Cost).
astar_queue([_-node(Node, Path, Cost) | Queue], Goal, ResultPath, ResultCost) :-
    findall(F-node(Next, [Next | Path], NewCost),
            (connected(Node, Next, StepCost),
             \+ member(Next, Path),
             NewCost is Cost + StepCost,
             heuristic(Next, H),
             F is NewCost + H),
            Children),
    append(Queue, Children, Frontier),
    keysort(Frontier, OrderedFrontier),
    astar_queue(OrderedFrontier, Goal, ResultPath, ResultCost).

% ?- astar(a, g, Path, Cost).
```

Practical 9: Implement Hill Climbing for optimization problems.

```
% Maximize f(X) = 49 - (X - 7)^2 over integer values 0..20.
objective(X, Score) :-
    Score is 49 - (X - 7) * (X - 7).

neighbor(X, Next) :-
    Next is X - 1,
    Next >= 0.
neighbor(X, Next) :-
    Next is X + 1,
    Next =< 20.

best_neighbor(X, Best, BestScore) :-
    findall(Score-Candidate,
            (neighbor(X, Candidate), objective(Candidate, Score)),
            Candidates),
    keysort(Candidates, Ordered),
    last(Ordered, BestScore-Best).

hill_climb(Start, Optimum, Score) :-
    objective(Start, StartScore),
    climb(Start, StartScore, Optimum, Score).

climb(Current, CurrentScore, Optimum, Score) :-
    best_neighbor(Current, Next, NextScore),
    ( NextScore > CurrentScore ->
        climb(Next, NextScore, Optimum, Score)
    ;   Optimum = Current,
        Score = CurrentScore
    ).

% ?- hill_climb(0, Optimum, Score).
```

Practical 10: Solve an optimization problem using Simulated Annealing to escape local optima.

```
:- use_module(library(random)).

% Two peaks: X=3 is a local optimum; X=17 is the global optimum.
objective(X, Score) :-
    ( X =< 10 ->
        Score is 30 - 2 * (X - 3) * (X - 3)
    ;   Score is 40 - (X - 17) * (X - 17)
    ).

neighbor(X, Next) :-
    ( X =:= 0 -> Next = 1
    ; X =:= 20 -> Next = 19
    ; random_between(0, 1, Direction),
      (Direction =:= 0 -> Next is X - 1 ; Next is X + 1)
    ).

accept(Delta, _) :- Delta >= 0.
accept(Delta, Temperature) :-
```

```
    Delta < 0,
    Probability is exp(Delta / Temperature),
    Random is random_float,
    Random < Probability.

simulated_annealing(Start, BestX, BestScore) :-
    objective(Start, StartScore),
    anneal(500, 20.0, Start, StartScore, Start, StartScore, BestX, BestScore).

anneal(0, _, _, _, BestX, BestScore, BestX, BestScore).
anneal(Steps, Temperature, Current, CurrentScore, OldBest, OldBestScore,
       BestX, BestScore) :-
    Steps > 0,
    neighbor(Current, Next),
    objective(Next, NextScore),
    Delta is NextScore - CurrentScore,
    ( accept(Delta, Temperature) -> NewCurrent = Next, NewScore = NextScore
    ; NewCurrent = Current, NewScore = CurrentScore
    ),
    ( NewScore > OldBestScore -> NewBest = NewCurrent, NewBestScore = NewScore
    ; NewBest = OldBest, NewBestScore = OldBestScore
    ),
    NextSteps is Steps - 1,
    NextTemperature is Temperature * 0.98,
    anneal(NextSteps, NextTemperature, NewCurrent, NewScore,
           NewBest, NewBestScore, BestX, BestScore).

% ?- set_random(seed(42)), simulated_annealing(3, BestX, BestScore).
```

Practical 11: Represent a Map-Coloring problem as a CSP and solve it using backtracking search.

```
% Available colors
color(red).
color(green).
color(blue).

% Adjoining regions in the map
adjacent(wa, nt).  adjacent(wa, sa).
adjacent(nt, sa).  adjacent(nt, q).
adjacent(sa, q).   adjacent(sa, nsw).
adjacent(sa, v).   adjacent(q, nsw).
adjacent(nsw, v).

different(X, Y) :- X \= Y.

map_coloring([wa-WA, nt-NT, sa-SA, q-Q, nsw-NSW, v-V, t-T]) :-
    color(WA), color(NT), color(SA), color(Q),
    color(NSW), color(V), color(T),
    different(WA, NT), different(WA, SA),
    different(NT, SA), different(NT, Q),
    different(SA, Q), different(SA, NSW), different(SA, V),
    different(Q, NSW), different(NSW, V).

% ?- map_coloring(Solution).
```

Practical 12: Solve the N-Queens problem using backtracking to satisfy CSP constraints.

```
:- use_module(library(lists)).

% queens(N, Rows) returns one safe row position for each column 1..N.
queens(N, Rows) :-
    numlist(1, N, Candidates),
    permutation(Candidates, Rows),
    safe_queens(Rows).

safe_queens([]).
safe_queens([Row | OtherRows]) :-
    safe_from(Row, OtherRows, 1),
    safe_queens(OtherRows).

safe_from(_, [], _).
safe_from(Row, [OtherRow | RemainingRows], Distance) :-
    Row =\= OtherRow,
    abs(Row - OtherRow) =\= Distance,
    NextDistance is Distance + 1,
    safe_from(Row, RemainingRows, NextDistance).

% ?- queens(8, Rows).
```